Resources

Choose Command Syntax or Function Syntax

MATLAB® has two ways of calling functions, called *function syntax* and *command syntax*. This page discusses the differences between these syntax formats and how to avoid common mistakes associated with command syntax.

For introductory information on calling functions, see Calling Functions. For information related to defining functions, see Create Functions in Files.

Command Syntax and Function Syntax

In MATLAB, these statements are equivalent:

```
load durer.mat        % Command syntax
load('durer.mat')     % Function syntax
```

This equivalence is sometimes referred to as *command-function duality*.

All functions support this standard function syntax:

```
[output1, ..., outputM] = functionName(input1, ..., inputN)
```

In function syntax, inputs can be data, variables, and even MATLAB expressions. If an input is data, such as the numeric value `2` or the string array `["a" "b" "c"]`, MATLAB passes it to the function as-is. If an input is a variable MATLAB will pass the value assigned to it. If an input is an expression, like `2+2` or `sin(2*pi)`, MATLAB evaluates it first, and passes the result to the function. If the functions has outputs, you can assign them to variables as shown in the example syntax above.

Command syntax is simpler but more limited. To use it, separate inputs with spaces rather than commas, and do not enclose them in parentheses.

```
functionName input1 ... inputN
```

With command syntax, MATLAB passes all inputs as character vectors (that is, as if they were enclosed in single quotation marks) and does not assign outputs to user defined variables. If the function returns an output, it is assigned to the `ans` variable. To pass a data type other than a character vector, use the function syntax. To pass a value that contains a space, you have two options. One is to use function syntax. The other is to put single quotes around the value. Otherwise, MATLAB treats the space as splitting your value into multiple inputs.

If a value is assigned to a variable, you must use function syntax to pass the value to the function. Command syntax always passes inputs as character vectors and cannot pass variable values. For example, create a variable and call the `disp` function with function syntax to pass the value of the variable:

```
A = 123;
disp(A)
```

This code returns the expected result,

```
  123
```

You cannot use command syntax to pass the value of `A`, because this call

```
disp A
```

is equivalent to

```
disp('A')
```

and returns

```
A
```

Avoid Common Syntax Mistakes

Suppose that your workspace contains these variables:

```
filename = 'accounts.txt';
A = int8(1:8);
B = A;
```

The following table illustrates common misapplications of command syntax.

This Command...	Is Equivalent to...	Correct Syntax for Passing Value
`open filename`	`open('filename')`	`open(filename)`
`isequal A B`	`isequal('A','B')`	`isequal(A,B)`
`strcmp class(A) int8`	`strcmp('class(A)','int8')`	`strcmp(class(A),'int8')`
`cd tempdir`	`cd('tempdir')`	`cd(tempdir)`
`isnumeric 500`	`isnumeric('500')`	`isnumeric(500)`
`round 3.499`	`round('3.499')`, which is equivalent to `round([51 46 52 57 57])`	`round(3.499)`

`disp hello world`	`disp('hello','world')`	`disp('hello world')`
		or
		`disp 'hello world'`

`disp "string"`	`disp('"string"')`	`disp("string")`

Passing Variable Names

Some functions expect character vectors for variable names, such as `save`, `load`, `clear`, and `whos`. For example,

```
whos -file durer.mat X
```

requests information about variable `X` in the example file `durer.mat`. This command is equivalent to

```
whos('-file','durer.mat','X')
```

How MATLAB Recognizes Command Syntax

Consider the potentially ambiguous statement

```
ls ./d
```

This could be a call to the `ls` function with `'./d'` as its argument. It also could represent element-wise division on the array `ls`, using the variable `d` as the divisor.

If you issue this statement at the command line, MATLAB uses syntactic rules, the current workspace, and path to determine whether `ls` and `d` are functions or variables. However, some components, such as the Code Analyzer and the Editor/Debugger, operate without reference to the path or workspace. When you are using those components, MATLAB uses syntactic rules to determine whether an expression is a function call using command syntax.

In general, when MATLAB recognizes an identifier (which might name a function or a variable), it analyzes the characters that follow the identifier to determine the type of expression, as follows:

- An equal sign (=) implies assignment. For example:

  ```
  ls =d
  ```

- An open parenthesis after an identifier implies a function call. For example:

  ```
  ls('./d')
  ```

- Space after an identifier, but not after a potential operator, implies a function call using command syntax. For example:

  ```
  ls ./d
  ```

- Spaces on both sides of a potential operator, or no spaces on either side of the operator, imply an operation on variables. For example, these statements are equivalent:

  ```
  ls ./ d

  ls./d
  ```

Therefore, MATLAB treats the potentially ambiguous statement `ls ./d` as a call to the `ls` function using command syntax.

The best practice is to avoid defining variable names that conflict with common functions to prevent ambiguity and have consistent whitespace around operators or to call functions with explicit parentheses.

See Also

Calling Functions | Create Functions in Files